

Foundations for nighttime lights data analysis

Ayush Patnaik^{1*}, Ajay Shah^{1*}, and Susan Thomas^{1*}

¹ xKDR Forum, India * These authors contributed equally.

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) ↗
- [Repository](#) ↗
- [Archive](#) ↗

Editor: ↗

Submitted: 12 September 2026

Published: unpublished

License

Authors of papers retain copyright
and release the work under a
Creative Commons Attribution 4.0
International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

A remarkable development in alternative data is the use of satellites that measure nighttime light radiance. From the early 1990s, Nightlights (NL) has been used to measure economic activity. NL correlates well with standard measures of growth and development (Doll et al., 2006), which allows measures of economic growth where existing measures are either unreliable or lack the continuity and/or resolution of constant satellite observation.

Raw monthly NL files are not analysis-ready. They contain missing observations, negative values, outliers, background noise, and cloud-related attenuation (Jiang et al., 2017; Ma et al., 2014; Uprety et al., 2019). As a result, every research team must individually implement preprocessing, often with project-specific scripts. This creates entry barriers and weakens comparability and reproducibility across studies.

Statement of need

NighttimeLights.jl is a Julia package for cleaning and bias-correcting monthly nighttime lights from the Visible Infrared Imaging Radiometer Suite (VIIRS). It provides a transparent and reproducible workflow that implements established methods from the applied literature, including cloud-related attenuation correction (Gupta et al., 2017; Patnaik et al., 2021; Pearson, 2002).

The package addresses a practical gap: many papers using NL data rely on individual preprocessing pipelines, making it difficult to reproduce findings or compare outcomes across studies. By consolidating key cleaning steps into one standard package, NighttimeLights.jl reduces duplicated effort and standardizes core data preparation.

State of the field

There are two dominant nighttime-lights products used in applied work: annual DMSP-OLS composites (1992-2013) and monthly VIIRS composites (2012-present). VIIRS has finer temporal and spatial resolution and is now the primary data source for contemporary analysis.

Most empirical NL workflows use combinations of missing-value recoding, outlier removal, and cloud-aware filtering, but implementation details vary across projects. This heterogeneity limits comparability. NighttimeLights.jl standardizes these operations while retaining modular control for users.

Several software tools already serve the nighttime-lights community, but they address a different part of the workflow. blackmarbler (Marty & Stefanini Vicente, 2024) and its Python counterpart BlackMarblePy (Stefanini Vicente & Marty, 2024) download NASA's Black Marble product and compute zonal statistics for administrative regions; Nightlights (Njuguna, 2019) does the same for the NOAA DMSP-OLS and VIIRS composites; and the VIIRS collections in Google Earth Engine (Gorelick et al., 2017) provide server-side access

39 and aggregation. These tools focus on acquisition and spatial aggregation, and the Black
40 Marble product applies sensor, terrain, and atmospheric corrections upstream at the pixel
41 level. None of them implement a configurable statistical cleaning pipeline for the raw NOAA
42 monthly composites, and none correct the cloud-cover attenuation bias that dominates monthly
43 series in monsoonal regions. Contributing these routines to the existing packages would
44 mean maintaining the same methods across three language ecosystems; providing them as
45 a standalone Julia package, with the cleaning pipeline itself as the unit of reuse, keeps the
46 methods in one place and keeps a cleaned panel reproducible from a single package version.

47 Software design

48 `NighttimeLights.jl` is implemented in Julia and built on `Rasters.jl` for geospatial data
49 handling. Data is represented as raster data cubes with dimensions (x, y, t) , and cleaning is
50 organized as composable functions. Core functions include:

- 51 ■ Missing-data handling: `na_recode`, `na_interp_linear`
- 52 ■ Negative-value handling: `replace_negative`
- 53 ■ Outlier treatment: `outlier_hampel`, `outlier_variance`
- 54 ■ Background-noise filtering: `bgnoise_PSTT2021`
- 55 ■ Cloud-bias correction: `bias_PSTT2021`

56 These are wrapped into high-level pipelines such as `PSTT2021()`, `PSTT2021_conventional()`,
57 and `clean_complete()`. This design supports both method-level customization and
58 standardized cleaning pipelines.

59 Several design choices shaped the package. Primarily, cleaning is exposed as a set of small
60 composable functions rather than a single opaque routine. This enlarges the API surface and
61 requires the user to respect an ordering between steps, but it allows for customization and lets
62 each stage be run, inspected, and switched out independently, which is what makes the package
63 usable for methodological work while still allowing reproducibility. Secondly, the package is
64 written in Julia because the core operations are per-pixel time-series computations over large
65 pixel counts. Hampel filters, smoothing-spline fits, linear interpolation are awkward to express
66 as fast vectorised array code and would otherwise require C extensions, if not for Julia's loop
67 performance. Finally, existing best-practices cleaning and cloud-bias-corrected cleaning are
68 kept as separately named and dated pipelines (`PSTT2021_conventional` and `PSTT2021`) so that
69 results remain reproducible as well as directly comparable to the large body of prior work that
70 does not correct for cloud cover.

71 Research impact statement

72 The package has been used as infrastructure for empirical work where cloud cover and data
73 quality strongly affect inference from nighttime lights. Its main impact is methodological
74 consistency: researchers can move from raw files to analysis-ready panels using explicit, shared
75 preprocessing steps.

76 Because each stage is exposed as a public function, `NighttimeLights.jl` also serves as a
77 platform for methodological development. Users can inspect intermediate outputs, compare
78 alternative cleaning choices, and contribute new preprocessing methods while preserving
79 reproducible workflows.

80 The cloud-bias correction implemented here was developed and validated in Patnaik et al.
81 (2021), and `NighttimeLights.jl` is the reference implementation of that method. The package
82 has been used to build analysis-ready VIIRS panels for applied economic work, including a
83 study of wartime economic activity in Ukraine (Hande et al., 2025) and an analysis of regional
84 economic patterns across Java, Indonesia (Aviliani et al., 2026). It also underpins the public
85 India Built and Lit dataset, which distributes cleaned nighttime-lights panels for India (xKDR

86 [Forum, 2026](#)). Reproducibility is supported concretely: a Mumbai example cube ships with
 87 the package and is loaded with `load_example()`, every pipeline stage is a documented public
 88 function, the package is registered in the Julia General registry, and the results quoted in this
 89 paper were produced with the pinned versions listed under Computational details.

90 Computational details

91 Results referenced in this manuscript were generated with Julia 1.12.6 and `NighttimeLights.jl`
 92 0.9.1. Julia is available at <https://julialang.org/> and the package is available at <https://github.com/xKDR/NighttimeLights.jl>.
 93

94 Installation from the General registry:

95] add NighttimeLights

96 Usage example

97 The following Julia code shows how to go from raw to cleaned data with `NighttimeLights.jl`.
 98 The example uses a rectangle of nighttime lights around Mumbai, which ships with the package
 99 and can be loaded with `load_example()`. Two inputs are needed. Both are Raster data cubes
 100 with dimensions (x, y, t) , where t indexes months:

- 101 ■ `mumbai_radiance`: the monthly mean radiance of each pixel, in $nW\ cm^{-2}\ sr^{-1}$.
- 102 ■ `mumbai_n_cloudfree`: the number of cloud-free observations that went into each monthly
 103 mean.

104 The simplest path is a single call:

```
mumbai_cleaned = clean_complete(mumbai_radiance, mumbai_n_cloudfree, bgthreshold=4)
```

105 which is presently identical to:

```
mumbai_cleaned = PSTT2021(mumbai_radiance, mumbai_n_cloudfree, bgthreshold=4)
```

106 Both wrap the sequence of steps below. Each step is a public function, so the sequence can
 107 be run one step at a time, inspected, or modified:

```
# 1. Missing data: months with no cloud-free observations become 'missing'
radiance_na = na_recode(mumbai_radiance, mumbai_n_cloudfree)
```

```
# 2. Negative values: recoded as 'missing'
radiance_nonneg = replace_negative(radiance_na)
```

```
# 3. Background noise: mask out pixels that are dark in the annual image
lit_mask = bgnoise_PSTT2021(radiance_nonneg, mumbai_n_cloudfree, 4)
radiance_lit = apply_mask(radiance_nonneg, lit_mask)
```

```
# 4. Cross-sectional outliers: mask out pixels with extreme variance over time
stable_mask = outlier_variance(radiance_lit, lit_mask)
radiance_stable = apply_mask(radiance_lit, stable_mask)
```

```
# 5. Time-series outliers: Hampel filter on each pixel's time series
radiance_despiked = long_apply(outlier_hampel, radiance_stable)
```

```
# 6. Cloud bias: correct attenuation in months with few cloud-free observations
pixel_mask = lit_mask .* stable_mask
radiance_corrected = bias_PSTT2021(radiance_despiked, mumbai_n_cloudfree, pixel_mask)
```

7. Gaps: linearly interpolate the remaining `missing` values

mumbai_cleaned = long_apply(na_interp_linear, radiance_corrected)

Two helpers recur in the sequence. `apply_mask` multiplies every monthly image in a data cube by a 2D mask, so that pixels marked as dropped in the mask are removed from every month. `long_apply` applies a function written for a single pixel's time series to every pixel of a data cube. The steps are:

1. **Missing data.** NOAA reports a radiance of 0 for a pixel-month in which every day was cloudy, which is indistinguishable from a genuinely dark pixel. `na_recode` uses `mumbai_n_cloudfree` to find pixel-months with zero cloud-free observations and recodes their radiance as missing.
2. **Negative values.** Radiance cannot be negative. Negative values in the raw data arise from calibration in the presence of airglow (Uprety et al., 2019). `replace_negative` recodes them as missing.
3. **Background noise.** Pixels with no lights, such as oceans, forests, and deserts, still show a small positive radiance. `bgnoise_PSTT2021` builds an annual image from the last twelve months of data, weighting each month by its number of cloud-free observations (Gupta et al., 2017). Pixels whose annual radiance falls below the threshold (here $4 \text{ nW cm}^{-2} \text{ sr}^{-1}$) are classified as dark. The result, `lit_mask`, is a 2D mask that is 1 for lit pixels and missing for dark ones. Applying it removes the dark pixels from every month.
4. **Cross-sectional outliers.** Fires, gas flares, and similar sources produce pixels whose radiance is far above anything from human settlement, and whose time series is highly variable. `outlier_variance` computes the standard deviation of each pixel's detrended time series and marks the top 0.1% of lit pixels as unstable. The result, `stable_mask`, is applied in the same way as `lit_mask`.
5. **Time-series outliers.** A pixel that is stable overall can still contain isolated spikes. `outlier_hampel` runs a Hampel filter (Pearson, 2002) over a single time series, with a window of 5 months and a cutoff of 3 standard deviations, and recodes flagged observations as missing. `long_apply` runs it over every pixel.
6. **Cloud bias.** Months with few cloud-free observations show attenuated radiance. This is the monsoon dip visible in the raw series of the figure below. For each pixel, `bias_PSTT2021` fits a smoothing spline of detrended radiance on the number of cloud-free observations, and raises observations from cloudy months to the level the spline predicts for clear months (Patnaik et al., 2021). The combined `pixel_mask` restricts the correction to pixels that survived steps 3 and 4.
7. **Remaining gaps.** `na_interp_linear` fills missing values in a time series by linear interpolation between the nearest observed values. Leading and trailing gaps take the nearest observed value. Applied over every pixel, this produces the complete data cube `mumbai_cleaned`.

Steps 1 to 5 and step 7, without the bias correction, are what most applied work already does. They are termed conventional cleaning in Patnaik et al. (2021) and are available as `PSTT2021_conventional()`. All three pathways, `clean_complete()`, `PSTT2021()`, and the explicit sequence, apply identical methods to transform the raw inputs into `mumbai_cleaned`.

149 An example

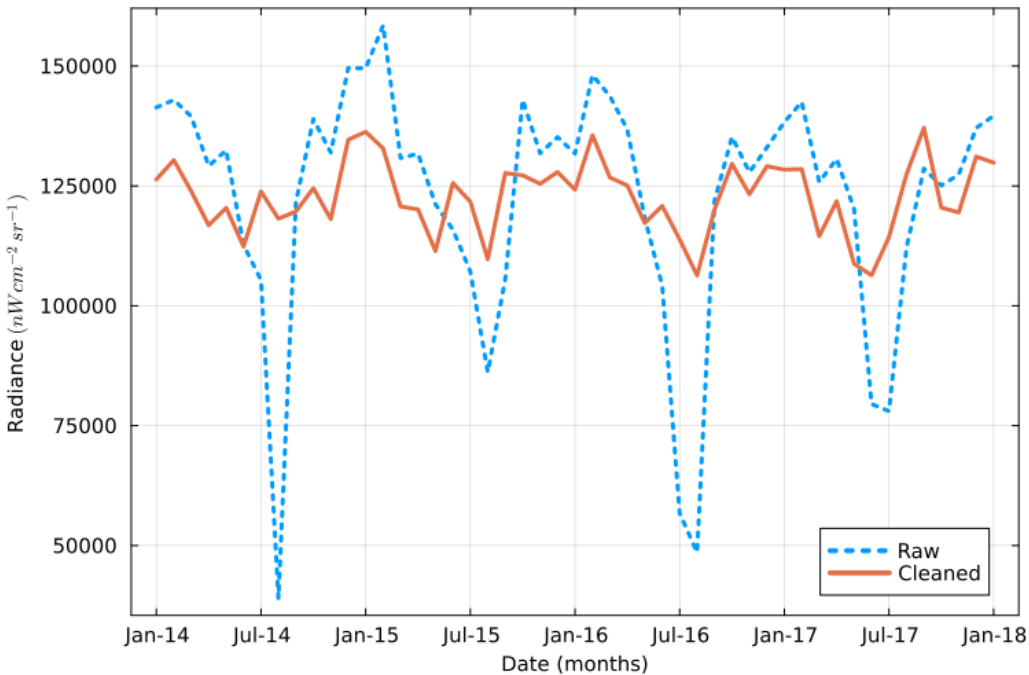


Figure 1: Aggregate radiance of raw and cleaned nighttime lights for Mumbai, January 2014 to January 2018.

Figure 1 shows the aggregate radiance of raw and cleaned NL of Mumbai. The notebook that produced this data can be found at https://github.com/xKDR/NighttimeLights.jl. On the y axis, the lines show the aggregate raw and cleaned NL measured in nW cm⁻² sr⁻¹. The x axis shows time measured in months from January 2014 to January 2018. Note the bias during the monsoon months (e.g. July), which is significantly reduced in the cleaned data.

Table 1: Effect of cleaning on nighttime lights for Mumbai.

Statistic	Raw	Cleaned
Minimum	-0.14	0.0
Mean	9.1	9.1
Median	3.2	0.0
99th percentile	54.0	59.0
Maximum	4300.0	1700.0

The table shows some summary statistics for raw and cleaned NL for Mumbai. Note the absence of negative values in the datacube. The maximum value after is also lower, as some outlier values have been removed. Half the values in the raw data cube are below 4, these can be attributed to background noise. All these measurements have been zeroed. The increase in the 99th percentile value is due to the bias correction procedure.

AI usage disclosure

No generative AI tools were used in the development of the software package, or the written text. AI assistance was used in formatting tables in the markdown document.

Acknowledgements

We acknowledge contributions from Anshul Tayal, Siddhant Chaudhary, and Hrishikesh Saikia to the codebase and package design. This work received financial support from the MIT Julia Lab.

References

- Aviliani, A., Kartawinata, M., Herawan, J. E., Parningotan, F., & Komala, A. (2026). When lights tell stories: The untold economic pattern of Java island. *Jurnal Ekonomi Indonesia*, 15(1), 1–18. <https://doi.org/10.52813/jei.v15i1.786>
- Doll, C. N., Muller, J.-P., & Morley, J. G. (2006). Mapping regional economic activity from night-time light satellite imagery. *Ecological Economics*, 57(1), 75–92. <https://doi.org/10.1016/j.ecolecon.2005.03.007>
- Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D., & Moore, R. (2017). Google Earth Engine: Planetary-scale geospatial analysis for everyone. *Remote Sensing of Environment*, 202, 18–27. <https://doi.org/10.1016/j.rse.2017.06.031>
- Gupta, P. K., Srivastav, S. K., Sessa Sai, M. V. R., Gharai, B., Senthil Kumar, A., & Krishna Murthy, Y. V. N. (2017). Analysis of SNPP-VIIRS-DNB derived nightlights over India. *38th Asian Conference on Remote Sensing: Space Applications — Touching Human Lives*.
- Hande, R., Patnaik, A., Shah, A., & Thomas, S. (2025). *Shedding light on the Russia-Ukraine war* (No. 40). xKDR Forum. <https://www.xkdr.org/paper/shedding-light-on-the-russia-ukraine-war>
- Jiang, W., He, G., Long, T., & Liu, H. (2017). Ongoing conflict makes yemen dark: From the perspective of nighttime light. *Remote Sensing*, 9(8). <https://doi.org/10.3390/rs9080798>
- Ma, T., Zhou, C., Pei, T., Haynie, S., & Fan, J. (2014). Responses of suomi-NPP VIIRS-derived nighttime lights to socioeconomic activity in china's cities. *Remote Sensing Letters*, 5(2), 165–174. <https://doi.org/10.1080/2150704x.2014.890758>
- Marty, R., & Stefanini Vicente, G. (2024). *blackmarbler: Black Marble data and statistics*. R package, Comprehensive R Archive Network. <https://doi.org/10.32614/CRAN.package.blackmarbler>
- Njuguna, C. (2019). *Rnightlights: Satellite nightlight data extraction*. R package, Comprehensive R Archive Network (archived).
- Patnaik, A., Shah, A., Tayal, A., & Thomas, S. (2021). *But clouds got in my way: Bias and bias correction of viirs nighttime lights data in the presence of clouds* (No. 7). xKDR Forum. <https://doi.org/10.2139/ssrn.3957319>
- Pearson, R. K. (2002). Outliers in process modeling and identification. *IEEE Transactions on Control Systems Technology*, 10(1), 55–63. <https://doi.org/10.1109/87.974338>
- Stefanini Vicente, G., & Marty, R. (2024). *BlackMarblePy: Georeferenced rasters and statistics of nighttime lights from NASA Black Marble*. Python package. <https://doi.org/10.5281/zenodo.10667925>
- Upreti, S., Cao, C., Gu, Y., Shao, X., Blonski, S., & Zhang, B. (2019). Calibration improvements in s-NPP VIIRS DNB sensor data record using version 2 reprocessing. *IEEE Transactions on Geoscience and Remote Sensing*, 57(12), 9602–9611. <https://doi.org/10.1109/tgrs.2019.2927942>
- xKDR Forum. (2026). *NighttimeLights and built-up volume dataset*. <https://xkdr.github.io/India-Built-and-Lit/>.